Jenkins task

MCQs Questions

1️⃣ What is Jenkins mainly used for?

A) Server Monitoring

B) Continuous Integration and Continuous Delivery

C) Database Management

D) Container Orchestration

2️⃣ Which type of job allows you to define build steps using code in Jenkins?

A) Freestyle Project

B) Pipeline Project

C) Multi-Configuration Project

D) External Job

3️⃣ Which file is used to define a pipeline in Jenkins?

A) pipeline.yaml

B) dockerfile

C) Jenkinsfile

D) build.gradle

4️⃣ What is the purpose of a Jenkins Agent (Node)?

A) To store source code

B) To execute jobs assigned by the Jenkins controller

C) To manage plugins

D) To configure webhooks

5️⃣ Which plugin is required to connect Jenkins with GitHub?

A) Docker Plugin

B) Git Plugin

C) Kubernetes Plugin

D) Maven Plugin

6️⃣ What is the purpose of a Webhook in Jenkins CI/CD?

A) To install plugins

B) To trigger build automatically on code push

C) To secure Jenkins server

D) To restart Jenkins

7️⃣ Which command is used inside Jenkins Pipeline to execute shell commands?

A) bash

B) cmd

C) sh

D) run

8️⃣ What is the purpose of `post` block in Jenkins Pipeline?

A) Define environment variables

B) Execute steps after pipeline stages

C) Define agents

D) Install plugins

9️⃣ What is the use of `sshagent` in Jenkins Pipeline?

A) Install SSH on server

B) Store SSH keys

C) Use stored SSH credentials during execution

D) Restart SSH service

🔟 What happens if a stage fails in Jenkins Pipeline (by default)?

A) The pipeline continues to next stage

B) The pipeline stops execution

C) Jenkins restarts automatically

D) All stages are skipped but marked successful

Scenario Based questions

You are hired as a **Junior DevOps Engineer** in a startup fitness company called **FitLife Gym**.

The development team has created a static HTML website for the gym.

Your manager has assigned you the following tasks:

1. Set up CI/CD infrastructure.
2. Host the website on a target server.
3. Ensure automatic deployment when code is pushed to GitHub.
4. Troubleshoot and fix pipeline issues if deployment fails.

You are responsible for complete automation.

Task 1: Infrastructure Setup

Create two Linux servers:

1. **Jenkins Server**
    - Install Java
    - Install Jenkins
    - Configure required plugins
    - Configure SSH credentials

2. **Target Server**
   - Install nginx
   - Ensure port 80 is open
   - Website should be accessible via browser

Task 2: Create GitHub Repository

1. Create a new public GitHub repository:

   ```
   gym-static-website
   ```

2. Add the following HTML code:

Simple Gym HTML Website Code

```html
<!DOCTYPE html>
<html>
<head>
    <title>FitLife Gym</title>
    <style>
        body {
            font-family: Arial;
            text-align: center;
            background-color: #111;
            color: white;
        }
        .container {
            margin-top: 100px;
        }
        h1 {
            color: #00ff99;
        }
        button {
```

```
            padding: 10px 20px;
            background-color: #00ff99;
            border: none;
            cursor: pointer;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>Hello from FitLife Gym</h1>
        <p>Transform Your Body. Transform Your Life.</p>
        <button>Join Now</button>
    </div>
</body>
</html>
```

Push this code to GitHub.

Task 3: Make Required Change

Modify the website:

Change:

```
Hello from FitLife Gym
```

To:

```
Hi from FitLife Gym
```

Push the changes to GitHub.

Task 4: Configure Webhook

- Configure GitHub webhook
- Trigger Jenkins job automatically on push

Task 5: Jenkins Pipeline

Create a Pipeline Job and add the following Jenkinsfile:

Students must troubleshoot and fix it.

Jenkinsfile

```
pipeline {
    agent any

    environment {
        SERVER_IP      = '172.31.1.200'
        SSH_CREDENTIAL = 'gym-key'
        REPO_URL       = 'https://github.com/your-username/gy
m-static-website.git'
        BRANCH         = 'main'
        REMOTE_USER    = 'ubuntu'
        REMOTE_PATH    = '/var/www/html'
    }

    stages {

        stage('Clone Repository') {
            steps {
                git branch: "${BRANCH}", url: "${REPO_URL}"
            }
        }

        stage('Deploy to Target Server') {
            steps {
                sshagent(['wrong-credential-id']) {
                    sh """
                        ssh -o StrictHostKeyChecking=no ${REM
```

```
OTE_USER}@${SERVER_IP} 'mkdir -p ${REMOTE_PATH}'
                        scp -o StrictHostKeyChecking=no -r *
${REMOTE_USER}@${SERVER_IP}:${REMOTE_PATH}
                    """
                }
            }
        }

        stage('Restart Nginx') {
            steps {
                sshagent([SSH_CREDENTIAL]) {
                    sh """
                        ssh -o StrictHostKeyChecking=no ${REM
OTE_USER}@${SERVER_IP} '
                            sudo systemctl restart nginxx
                        '
                    """
                }
            }
        }
    }

    post {
        success {
            echo 'Deployment Successful'
        }
        failure {
            echo 'Deployment Failed'
        }
    }
}
```